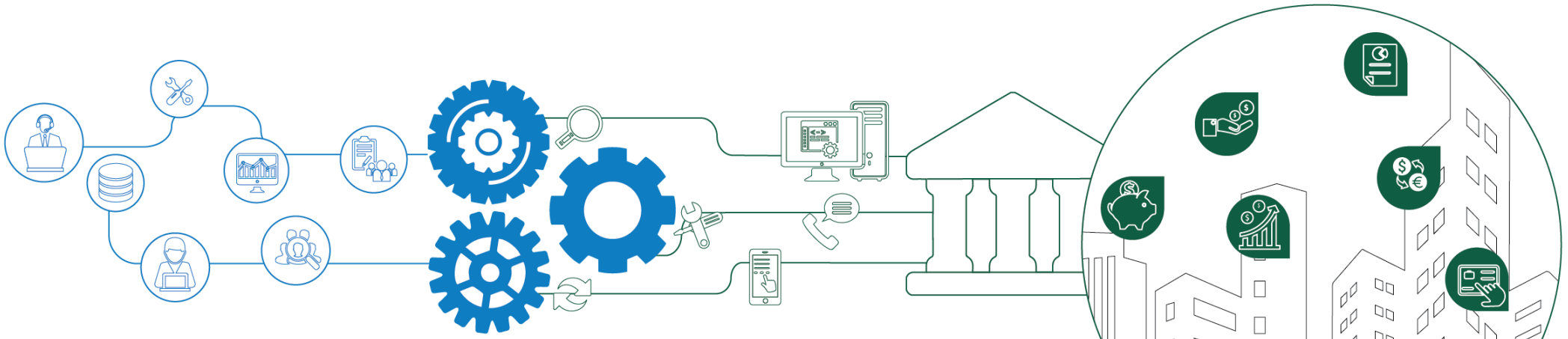


Mainframe Modernization- GCP BigQuery Approach

Agenda

- Mainframes to Cloud Migration Objectives
- Possible Solution and Recommendation
- BigQuery vs DataFlow Comparison
- Why BigQuery and BigQuery Architecture
- Target Architecture
- Benefits and Outcomes

Program Migration Objectives

To modernize the bank's legacy mainframe systems by transitioning to a cloud-native architecture, leveraging microservices for real-time applications and scalable cloud-based solutions for batch processing, with the goal of enhancing agility, reducing operational costs, improving customer experience, and ensuring regulatory compliance.

Business Outcomes

- Cost Reduction
- Improved Time-to-Market
- Enhanced Customer Experience
- Reusability
- Scalability and Flexibility
- Regulatory Compliance and Security
- Business Continuity and Resilience

Piloted Possible Solutions

1. General Batch processing

✅ **Spring Batch/Python**

Develop code in Java or Python for batch processing and use EDA(with MQ) for real time events

2. GCP Approach

✅ **ETL Approach**

Transformation using Apache Beam with Dataflow (Supports both Offline Batch and Real time) and load the data to Database(CloudSQL, BigQuery)

✅ **ELT Approach**

Load the data to BigQuery and heavy transformation on BigQuery using Stored Procedures and Orchestration with Apache Airflow.

Comparing BigQuery vs Dataflow(Apache Beam)

Feature	BigQuery Approach	Dataflow Approach
Best For	High-volume, simple batch loading (EL)	Complex ETL/ELT pipelines, real-time streaming, and transformations
Effort to Implement	Low (simple SQL command or UI operation)	High (requires coding a Beam pipeline)
Cost	Low (free from GCS, pay for storage)	Higher (pay for worker resources and processing time)
Transformation	Minimal (SELECT on load)	Powerful and extensive
Flexibility	Low	High
Ideal Use Case	Loading raw logs, bulk CSV/JSON files, database dumps	Building a data warehouse, real-time analytics, or complex data processing

Why BigQuery

When migrating data pipelines or analytics workloads to GCP, BigQuery offers several advantages:

✅ **1. Scalability and Performance**

Handles **petabyte-scale** datasets with **automatic scaling**.

Uses **columnar storage** and **Dremel execution engine** for fast query performance.

✅ **2. Serverless Architecture**

No infrastructure to manage—just load data and query.

Automatically handles provisioning, scaling, and optimization.

✅ **3. Flexible Ingestion Options**

Supports **batch loads**, **streaming inserts**, and **federated queries** from sources like Cloud Storage, Cloud SQL, or external databases.

Easily integrates with **Dataflow**, **Pub/Sub**, and **Apache Beam** for ETL/ELT pipelines.

✅ **4. Powerful SQL-Based Transformations**

Ideal for **ELT** workflows where raw data is loaded first and transformed using SQL.

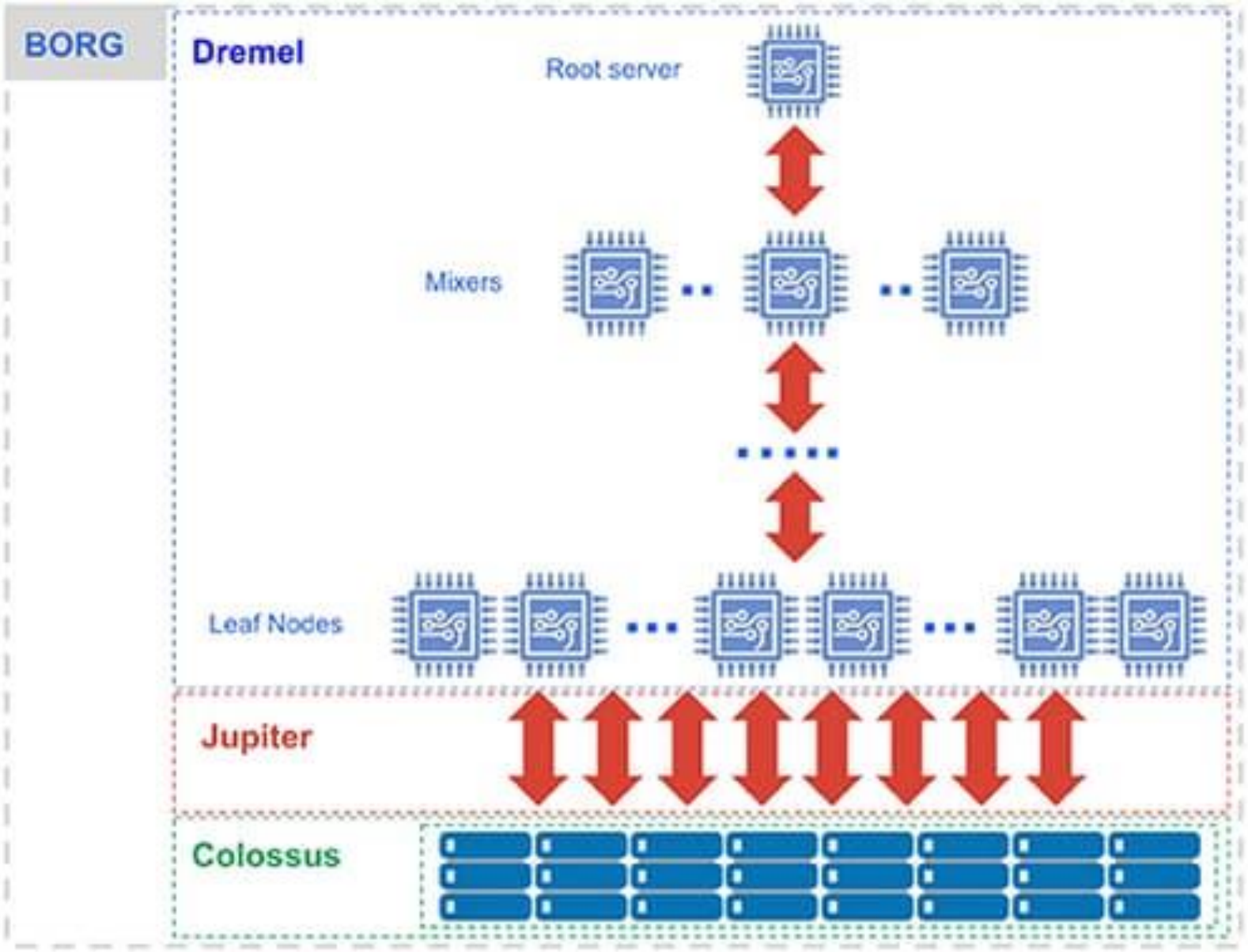
Supports **nested and repeated fields**, **window functions**, **joins**, and **CTEs**.

✅ **5. Cost Efficiency**

Pay-per-query model or **flat-rate pricing**.

Can optimize costs using **partitioned** and **clustered tables**, **materialized views**, and **query caching.**

BigQuery - Architecture



Capacitor — The Storage Format

Colossus — The Storage System

Dremel — The Query Execution Engine

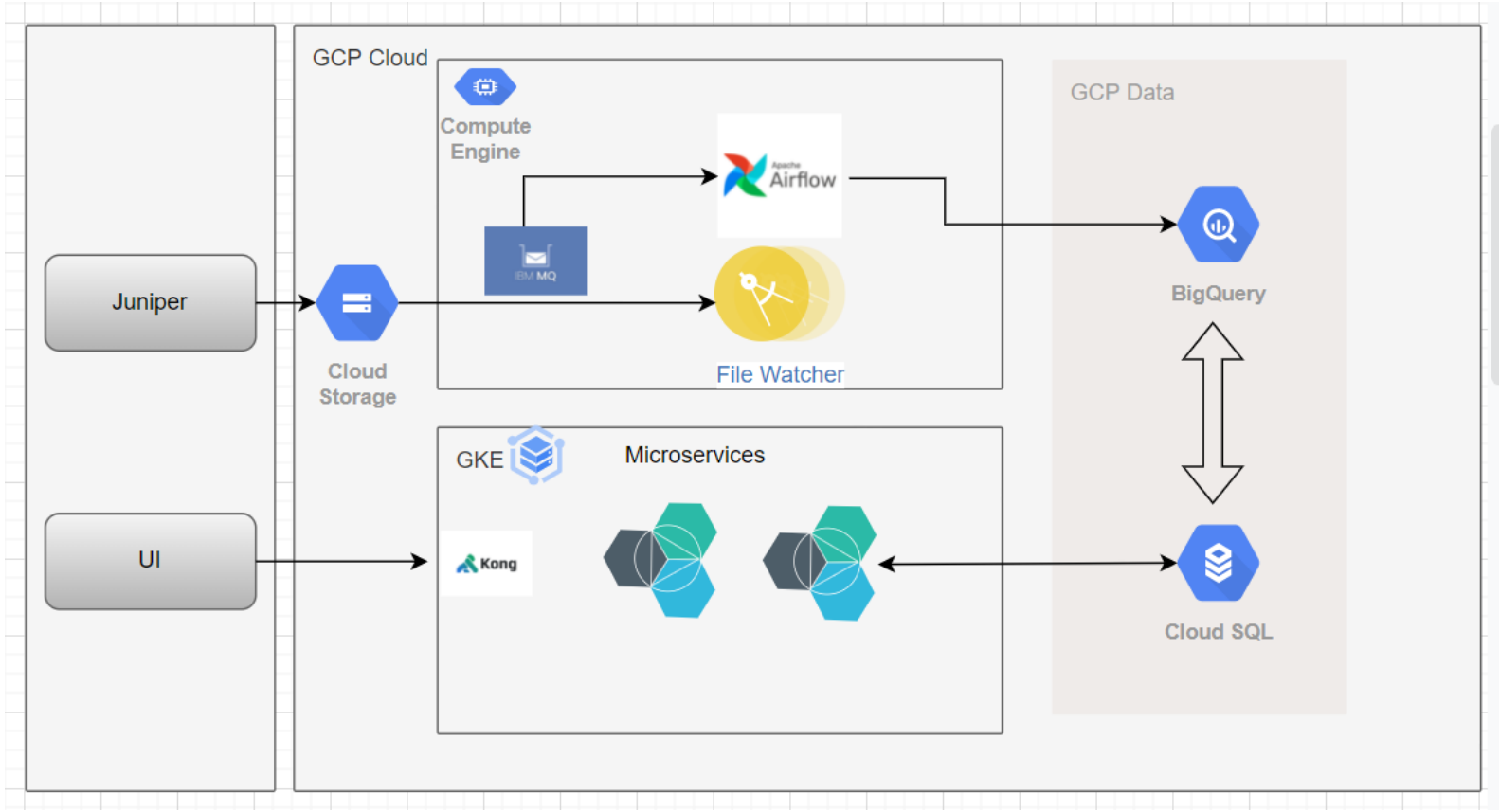
Borg — The Compute Manager

Jupiter — The Network Backbone

BigQuery - How a Query or Stored Procedure Runs in BigQuery

- **Query Submission :** You submit a query, like *` SELECT * FROM KubeCon.Hall1 WHERE attendance > 30.*
- **Root Node Processing :** The root node of Dremel receives the query and reads metadata to understand the table structure.
- **Task Distribution :** The query is divided into smaller tasks, which are sent to intermediate nodes.
- **Optimization :** Intermediate nodes optimize the tasks, determining the best way to execute them (e.g., which data partitions to read).
- **Data Retrieval and Computation :** Leaf nodes read the necessary data from Colossus, apply filters, and perform calculations.
- **Aggregation :** Results are aggregated by intermediate nodes and sent back to the root node.
- **Final Result :** The root node compiles the final result and returns it to you.

High Level Architecture



Technology Stack

- *Springboot Microservices*
- *Java and Python*
- *Kong API Gateway*
- *Angular Frontend*
- *BigQuery*
- *GKE Platform*
- *Cloud SQL(Postgres)*
- *Apache Airflow and Rabbit MQ*
- *Prometheus and Grafana for Observability.*

Architecture Explanation

Data Ingestion:
Files from Different Sources will be ingested into Cloud Storage Bucket using secured file transfer mechanisms. File Watcher is set up for each file, Once all the files are received, then actual processing starts.

Data Transformation:
RAW Layer: Where the initial load happens from Storage Bucket to DB. Data Validations and DQ Rules are applied in this layer.
STAGE Layer: Data is loaded to stage post validations and will be used as backup layer.
PROCESSED Layer: BigQuery Stored Procedures will read the data from RAW/STAGE Layer and does the transformation and populates the data into PROCESSED and FINAL Layer.

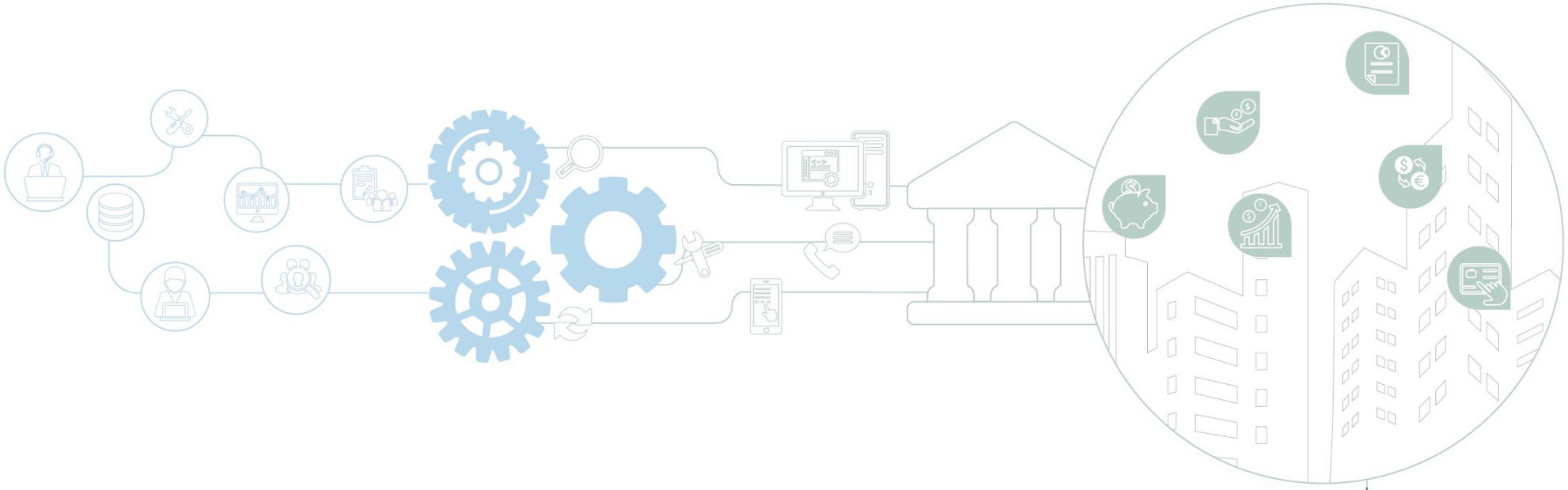
Data Replication:
Processed Data will be copied to Postgres Stage Layer and once successful will be copied to main schema. During the data replication stage, UI will not be allowed to perform any write operations with a maintenance message.

Key Benefits and Outcomes

- Reduced fair amount of cost by BigQuery Approach.
- Able to process 3 Million Record in 2 Minutes when compared with Dataflow which is 20 mins for 3 Million Records.
- Easy to implement transformation logic through stored procedures than writing complex Java/Apache Beam code.
- Easy to load/unload data between CloudSQL and Vice Versa.
- Highly available and highly Scalable for large workloads comes with InBuild Cache for low latencies.
- InBuild Metrics help Trouble Shooting Easier.
- Automating Jobs using Apache Airflow reduces manual errors on Database.

Challenges Faced

- Data Copy/Sync Issues from Bigquery to SQL.



THANK YOU

Appendix